État d'implémentation face à la gouvernance

> ***Statut : constat.*** *Ce document décrit ce qui a été vérifié dans le code, avec ses sources. Les objectifs et les plans sont dans* `cible/`*.*

> *Responsable : Cédric · Version 0.2 du 20/10/2026 · Relevé sur* `main` *après la fusion des PR #14 à #21 (plateforme, WAF, SIEM, durcissement, investigation, API derrière le WAF) et #24 (reconnaissance faciale)*
>
> *Les documents de gouvernance ont été écrits avant le code. Ce registre dit, pour chaque hypothèse importante, si le code la* ***confirme****, la* ***contredit*** *ou ne l'a* ***pas encore implémentée****. Il est à mettre à jour après chaque fusion importante et avant les essais (WOR-73).*

1. Ce qui existe au 20/10/2026

Composant	État	Où
API de paiement (Node.js) avec authentification par session, idempotence des paiements, métriques Prometheus, journaux JSON	Implémentée. **Dans le socle, elle garde tout en mémoire** (voir §2)	`app/backend/`
Front web et page d'administration	Implémentés	`app/front/`
Base PostgreSQL de l'application	Implémentée, **sans sauvegarde**, utilisée seulement par le déploiement `app/`	`app/infra/docker-compose.yml`, `app/infra/database/init/001_schema.sql`
Socle : Vault, PostgreSQL, Prometheus (collecte toutes les 5 s), Loki (rétention 7 jours), Grafana	Implémenté. L'API y est collectée (métriques et journaux)	`infra/docker-compose.yml`
Extraction des features de détection (fenêtre de 60 s, pas de 10 s)	Implémentée	`ml/features/extract.py`
Modèle de détection, envoi des alertes	**Absent**	`ml/training/`, `ml/inference/` vides
Moteur de remédiation	**Absent**	`services/remediation/README.md` : « À implémenter »
WAF ModSecurity + OWASP CRS, en mode bloquant, seul point d'entrée publié du socle	Implémenté, l'API est derrière lui. Les injections sont bloquées en 403. **Pas de rechargement à chaud** : une règle change par fichier, puis le WAF redémarre (`config_watch`). Pas de limitation de débit au niveau du WAF	`infra/docker-compose.yml`, `infra/waf/README.md`

Composant	État	Où
Tableau de bord SOC / SIEM dans Grafana : blocages du WAF, accès à Vault, flux corrélé WAF · Vault · PostgreSQL	Implémenté ; **aucune règle d'alerte**	`infra/grafana/provisioning/dashboards/soc-siem.json`
Tableau de bord d'incident (WOR-40)	**Absent** ; la page d'administration affiche des statuts fixes	`services/dashboard/`, `app/backend/api/src/main.ts`
Remise à l'état sain	Implémentée ; **efface base, journaux, métriques et Vault**	`scripts/reset-known-good.sh`
Export des journaux de Loki vers un dossier de preuves, avec empreintes SHA-256	Implémenté, **à lancer à la main** avant la remise à l'état sain (qui ne l'appelle pas)	`scripts/export-logs.sh`
Durcissement des conteneurs (non-root, `cap_drop`, limites de ressources)	Implémenté	`infra/docker-compose.yml`
Reconnaissance faciale pour la double authentification du back-office (WOR-95)	Implémentée, interne, ne stocke que des empreintes du visage	`services/face-recognition/`
Assistant RAG en lecture seule	Implémenté	`rag/docker-compose.rag.yml`
Scripts d'attaque A et B	Implémentés ; l'attaque B vise encore une route inexistante (voir §3)	`attack/`

Il existe encore **quatre déploiements Docker Compose séparés** (application, socle, labo IA, RAG). Le socle porte maintenant l'API derrière le WAF, mais sans base de données. Il manque toujours le détecteur et le moteur de remédiation : la chaîne attaque → détection → remédiation → vérification ne peut pas être jouée de bout en bout.

2. Hypothèses de la gouvernance confrontées au code

Point principal : dans le socle, l'API garde ses données en mémoire. Elle n'utilise PostgreSQL que si `STORAGE_DRIVER=postgres` (`app/backend/api/src/main.ts:42`), et cette variable n'est pas définie pour `payment-api` dans `infra/docker-compose.yml`. Conséquences :

- un simple redémarrage (première réponse au déni de service, fiche S1) efface toutes les transactions et toutes les sessions ;
- le RPO = 0 est impossible, même sans attaque ;
- une injection ne peut altérer aucune base, donc il n'y a rien à restaurer.

La correction est simple côté infrastructure (définir la variable et brancher l'API sur la base du socle), mais elle doit être faite avant les essais pour que les objectifs de `cible/rto-rpo.md` aient un sens.

Hypothèse	Document	Verdict	Constat
Aucune transaction confirmée perdue (RPO = 0)	`cible/rto-rpo.md`	**Contredit**	API en mémoire dans le socle : tout redémarrage perd les transactions. Ailleurs, aucune sauvegarde ; la remise à l'état sain repart des données de départ
Après une injection : snapshot puis rejeu des transactions légitimes	`cible/rto-rpo.md` §5, `cible/pca-pra.md` S2	**Non implémenté**	Pas de snapshot. Point toujours à trancher avec Michael et Reza
Deux signaux concordants avant une action destructrice	`cible/pca-pra.md` S4	**Non implémenté**	Ni moteur, ni modèle. Les runbooks parlent de deux fenêtres consécutives d'une même famille, ce qui n'est pas deux signaux indépendants
Au plus 3 remédiations en 10 minutes par service	`cible/pca-pra.md` §3	**Non implémenté**	Les runbooks prévoient une escalade après 2 tentatives (`docs/runbooks/reprise.md`) : à harmoniser
Journaux copiés avant toute restauration	`cible/pca-pra.md` S2, §6	**Partiellement confirmé**	`scripts/export-logs.sh` copie les journaux de Loki avec empreintes, mais il faut le lancer à la main : la remise à l'état sain ne l'appelle pas
Assistant RAG sans accès privilégié	`cible/pca-pra.md` §5	**Confirmé**	Conteneur en lecture seule, sans droits ni accès au moteur Docker
Hôte unique, aucune sauvegarde hors de l'hôte	`cible/nis2-dora.md`, `cible/pca-pra.md` S7	**Confirmé**	Inventaire Ansible local uniquement
Secrets dans HashiCorp Vault	`cible/ebios-rm.md` BS9	**Partiellement contredit**	Vrai pour le socle. L'application a son mot de passe de base en clair dans son fichier Compose, et sa clé d'API OpenAI en variable d'environnement
Collecte toutes les 5 s, détection en 20 à 70 s	`cible/rto-rpo.md` §4	**En partie confirmé**	Collecte à 5 s et fenêtre de 60 s confirmées. Le délai de détection ne peut pas être mesuré : il n'y a pas encore de détecteur
Une injection altère des données, qu'il faut ensuite restaurer	`cible/pca-pra.md` S2	**Contredit**	Le WAF bloque 9 vecteurs sur 10 de l'attaque B en 403 (`infra/waf/README.md`), et l'API du socle n'a pas de base. La protection est démontrée ; la restauration ne peut pas l'être
Vérification de la reprise par une transaction réelle	`cible/rto-rpo.md` §1	**À construire**	`/health` répond toujours « ok » sans rien vérifier : il ne peut pas servir de preuve de reprise

3. Nouveaux constats à intégrer à l'analyse de risques (WOR-67)

Constat	Risque concerné	Gravité du constat
Port de la base de l'application (55432) publié sur toutes les interfaces, avec un mot de passe en clair dans le dépôt	R5 exfiltration	**Élevée** : la base est joignable depuis le réseau `RESILIENCE-G6`
L'assistant de discussion de l'application envoie les messages des utilisateurs à OpenAI	Fuite de données (ER3), prestataire TIC tiers (DORA art. 28)	Moyenne : données fictives, mais incohérent avec un assistant « local »

Constat	Risque concerné	Gravité du constat
Code de vérification des paiements fixe (123456 par défaut) **et renvoyé dans la réponse de l'API** (`demoCode`)	R4 usurpation, R2 fraude	Moyenne en démo, critique en production : la vérification ne protège rien
Limitation de débit sur la connexion (12 essais / 15 min), mais pas de verrouillage de compte	R4 bourrage d'identifiants	Faible : la limitation freine déjà l'attaque
Vault avec une seule part de clé, clé de descellement et jeton root dans un fichier de l'hôte	R3, R7	Moyenne : acceptable pour un prototype, à citer en limite
Attaque A sur `/health` : l'API limite à 60 requêtes par minute, et comme elle ne lit pas l'adresse réelle du client derrière le WAF (pas de `trust proxy`), cette limite vaut pour **tous les clients réunis**. L'attaque produit des erreurs 429 pour tout le monde plutôt qu'une saturation	Validité des essais, R1 déni de service	**Élevée** : le déni de service est réel, mais il est causé par la limitation, pas par la saturation que la détection et la reprise sont censées traiter
Attaque B sur `/api/pay`, qui n'existe pas (les routes sont en `/v1/…`)	Validité des essais	**Élevée** : l'attaque B ne modifie aucune donnée, il n'y a rien à restaurer
Pas de solde ni de débit dans le modèle de données (intentions de paiement uniquement)	ER2 altération	Faible : le risque devient « paiements altérés ou dupliqués » ; l'idempotence protège déjà contre les doublons

4. Ce qu'il faut avant les essais de mercredi

Par ordre de priorité, pour que la démo prouve ce que la gouvernance affirme :

1. **Brancher l'API du socle sur PostgreSQL** (`STORAGE_DRIVER=postgres`) : sans cela, un redémarrage perd toutes les transactions. Infrastructure, Développement.
2. **Remplacer `/health` par une vérification réelle** : base joignable et transaction de test réussie. Développement.
3. **Construire le moteur de remédiation** (WOR-38), avec la vérification de la reprise. Développement.
4. **Entraîner le modèle et envoyer les alertes** (WOR-52, WOR-53). IA / Data.
5. **Recâbler les attaques A et B** : B sur une route en `/v1/…`, A en tenant compte de la limite de 60 requêtes par minute (activer `trust proxy` pour qu'elle s'applique par client). Pour l'attaque B, **décider ce qu'on démontre** : le blocage par le WAF (déjà obtenu), ou la restauration, qui suppose qu'une injection passe le WAF et écrive en base. Cybersécurité, Gouvernance.
6. **Une sauvegarde de la base avant chaque essai**, au minimum un `pg_dump` horodaté, conservé hors des volumes purgés. Infrastructure.
7. **Fermer le port 55432 et passer les secrets de l'application dans Vault.** Cybersécurité.
8. **Appeler `scripts/export-logs.sh` depuis la remise à l'état sain**, pour ne plus perdre les preuves des essais par oubli. Infrastructure.